



JULIAN KRAUSS · PROGRAMMIERTECHNIK

Bibliotheksverwaltung in C++17

Musterlösung zu Aufgabenblatt 52

Eine vollständige Konsolenanwendung mit drei Klassen, kontrollierter Ausleihe und automatischen Prüfungen. Die Erklärung führt vom Klassenmodell bis zur Abgabe auf GitHub.

Dokumentangabe	Wert
Name	Julian Krauß
Fach und Lehrkraft	Programmiertechnik · Dr. Michael Litera
Stand und Version	02.10.2026 · 1.0
Grundlage	Aufgabenblatt 52 · Teile 0 bis 5
Dateien	PDF · bearbeitbare Word-Datei · C++-Projekt
Werk-ID	JK-PT-52-2026-001

Was die Lösung enthält

Fünf Beispielmedien, zwei Mitglieder, alle geforderten Klassen und Operatoren sowie 29 automatische Prüfungen. Der vollständige Quellcode steht im Anhang und zusätzlich als kompilierbare Datei im Projektpaket.

Geprüft: Der Code kompiliert unter C++17 mit g++ 13.3.0 ohne Warnungen. Alle 29 Prüfungen sind bestanden.

KI-gestützte Lernvorlage. Vor der persönlichen Abgabe den Code nachvollziehen, die Reflexion anpassen und den eigenen Git-Arbeitsverlauf dokumentieren.

00 / AUFGABENBLATT 52

Inhalt

Die Einträge sind anklickbar. Die Kapitel erklären die Lösung; der Anhang enthält den vollständigen Inhalt von `src/main.cpp`.

01 Auftrag und Projektdateien	3
02 Klassen und Beziehungen	4
03 Ausleihen und Rückgabe	5
04 Suche und sichere Objektzugriffe	6
05 Bauen und Testergebnisse	7
06 Git und Abgabe	8
07 Reflexion	9
08 Abgabecheck und Quellen	10
09 Vollständiger Quellcode	11

So verwendest du das Projektpaket

Entpacke `bibliothek-cpp`, öffne den Ordner und führe den Kompilierbefehl aus Kapitel 05 aus. Lies zuerst die Erklärung zu den Klassen und prüfe anschließend die zugehörigen Stellen in `main.cpp`.

Für GitHub: `GIT_ABGABE.md` enthält die vollständige Schrittfolge. Die Musterlösung liefert keine bereits erledigten GitHub-Schritte und keine erfundene Commit-Historie.

Gestaltungsgrundlage: Corporate Identity & Design System JK-CD-DE-2026-003, PDF-Revision 3.2, Release 4.6.3. Für den Quellcode ist das Raster auf gut lesbare A4-Hochformatseiten angepasst.

01 / AUFGABENBLATT 52

Auftrag und Projektdaten

Die Bibliothek verwaltet Bücher, DVDs und Zeitschriften über eine gemeinsame Klasse Medium. Mitglieder dürfen höchstens drei Medien gleichzeitig ausleihen. Die Bibliothek verbindet beide Objektgruppen über eindeutige Kennungen.

Teil	Anforderung	Umsetzung
0	Repository und Projektstruktur	Dateien vorbereitet; GitHub-Schritte selbst ausführen.
1	Enum und Medium	Private Attribute, Prüfung, Getter, Status und Beschreibung.
2	Mitglied	Medien-IDs, Limit 3, Ausleihe und eigene Rückgabe.
3	Bibliothek und Operatoren	Wertespeicher, ID-Suche, Textsuche, << und ==.
4	Hauptprogramm und Tests	5 Medien, 2 Mitglieder, 29 automatische Prüfungen.
5	Reflexion und Abschluss	Antwortvorlage, Git-Anleitung und Abgabecheck.

Dateien im Projektordner

Datei	Aufgabe
src/main.cpp	Alle Klassen, freie Funktionen und main().
README.md	Beschreibung, Build-Befehle, Programmausgabe und Lernpunkte.
REFLEXION.md	Sechs Musterantworten zur persönlichen Überarbeitung.
.gitignore	Schließt Build-Ausgaben und IDE-Dateien aus.
GIT_ABGABE.md	Schritte für Commits, Branch, Merge, Issue, Klon und Tag.

Bewusste Festlegungen der Musterlösung

IDs und Jahreszahlen müssen positiv sein. Titel, Urheber und Mitgliedsnamen dürfen weder leer sein noch nur aus Leerzeichen bestehen. Auch Mitgliedsnummern sind eindeutig. Diese Prüfungen ergänzen die ausdrücklich geforderte Titelprüfung.

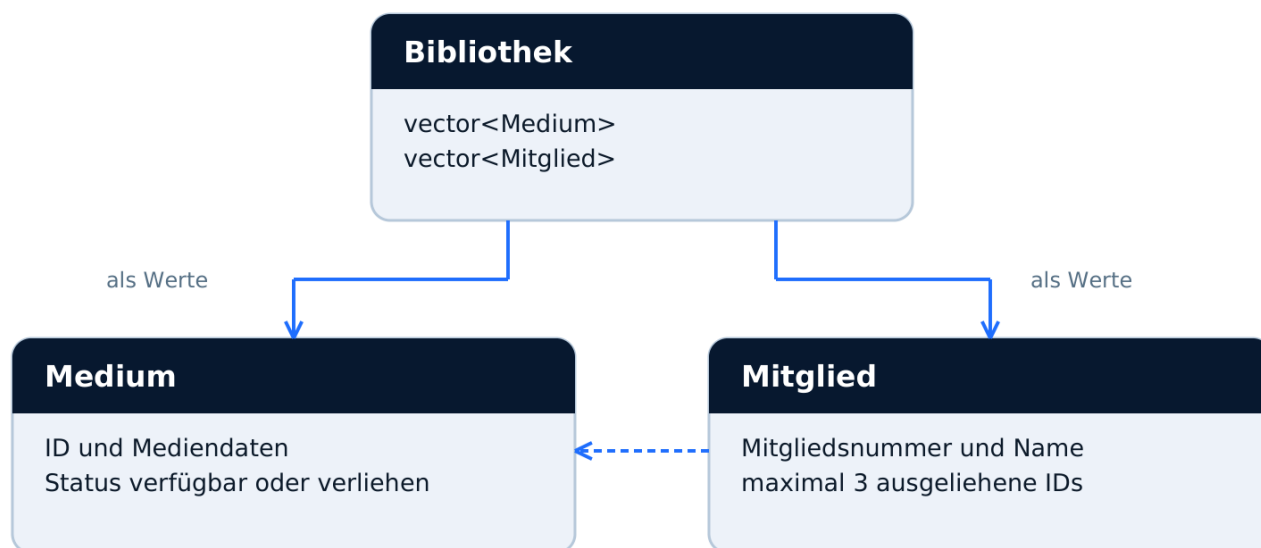
Die Textsuche beachtet Groß- und Kleinschreibung. Ein leerer Suchtext liefert alle Medien. Das Programm verwendet feste Beispieldaten und benötigt keine Tastatureingabe; ein Menü ist im Aufgabenblatt nicht gefordert.

Ein-Datei-Vorgabe: Die Reihenfolge lautet Medium, Mitglied, Bibliothek, main. Hilfsfunktionen und Operatoren stehen jeweils bei den zugehörigen Klassen.

02 / AUFGABENBLATT 52

Klassen und Beziehungen

Jede Klasse hat eine klar begrenzte Aufgabe. Medium kennt seine Daten und seinen Status. Mitglied kennt seine Ausleihen. Bibliothek verwaltet die Objekte und vermittelt zwischen ihnen.



Gestrichelter Pfeil: Mitglied merkt sich Medien-IDs, keine Medienzeiger.

Abbildung 1 Objektbeziehungen der Musterlösung · Gestaltung © 2026 Julian Krauß

Medium schützt seine Daten

Attribut	Typ	Bedeutung
id	int	Eindeutige Kennung des Mediums.
titel	std::string	Titel des Buchs, Films oder Hefts.
urheber	std::string	Autor, Regisseur oder Verlag.
jahr	int	Positive Jahreszahl.
typ	MediumTyp	Buch, DVD oder Zeitschrift.
verfuegbar	bool	Startwert true; Ausleihen setzt false.

Die Attribute sind private. Öffentliche Getter lesen sie als const-Methoden. Der Konstruktor belegt alle Attribute direkt über eine Initialisierungsliste und wirft bei ungültigen Eingaben `std::invalid_argument`.

Enum und Operatoren

`enum class MediumTyp` begrenzt die vorgesehenen Medientypen. `typAlsText()` liefert den Typnamen; `beschreibung()` ergänzt per switch den passenden Hinweis auf Autor, Regie oder Verlag.

`operator<<` gibt `beschreibung()` aus. `operator==` vergleicht ausschließlich die ID. Zwei Objekte mit gleicher ID gelten deshalb auch dann als gleich, wenn andere Daten abweichen.

03 / AUFGABENBLATT 52

Ausleihen und Rückgabe

Eine Ausleihe ist erst vollständig, wenn der Medienstatus und die Ausleihliste des Mitglieds übereinstimmen. Deshalb ruft main() die passenden Methoden der Bibliothek auf; diese finden die Objekte und leiten den Vorgang weiter.

Ablauf einer Ausleihe

1. Bibliothek sucht das Mitglied und das Medium anhand ihrer Nummern. Fehlt eines der Objekte, liefert sie false.
2. Mitglied prüft sein Limit und ob dieselbe Medien-ID bereits in seiner Liste steht.
3. Medium::ausleihen() prüft die Verfügbarkeit. Nur ein freies Medium wird auf verliehen gesetzt.
4. Bei Erfolg speichert das Mitglied die Medien-ID und liefert true zurück.

Die zentrale Methode

```
bool ausleihen(Medium& m) {
    if (ausgelieheneIds.size() >= 3 || hatAusgeliehen(m.getId())) {
        return false;
    }
    ausgelieheneIds.reserve(3);
    if (!m.ausleihen()) return false;
    ausgelieheneIds.push_back(m.getId());
    return true;
}
```

Medium& ist eine Referenz auf das tatsächlich gespeicherte Medium. Die Methode verändert dadurch das Original. reserve(3) beschafft den benötigten Speicher vor der Statusänderung, ohne die Anzahl der Ausleihen zu erhöhen.

Rückgabe nur durch das richtige Mitglied

Mitglied::zurueckgeben() sucht zuerst die Medien-ID in der eigenen Liste. Ist sie nicht enthalten, bleibt das Medium unverändert. Sonst ruft die Methode m.zurueckgeben() auf und entfernt die ID nur bei erfolgreicher Rückgabe.

Vorgang	Ergebnis
Medium frei und Limit nicht erreicht	true; Status und ID-Liste ändern sich.
Medium bereits verliehen oder Limit erreicht	false; keine neue Ausleihe entsteht.
Rückgabe eines nicht selbst geliehenen Mediums	false; die fremde Ausleihe bleibt bestehen.

Warum IDs? Beim Wachsen eines Vektors können sich Speicheradressen ändern. Die numerische ID eines Mediums bleibt dabei gleich.

04 / AUFGABENBLATT 52

Suche und sichere Objektzugriffe

Bibliothek besitzt `std::vector<Medium>` und `std::vector<Mitglied>`. Beide Vektoren speichern vollständige Objekte als Werte. IDs dienen als dauerhafte Kennungen innerhalb dieses Modells.

Methode	Verhalten
<code>mediumHinzufuegen(Medium)</code>	Lehnt eine bereits vorhandene Medien-ID mit einer Ausnahme ab.
<code>mitgliedHinzufuegen(Mitglied)</code>	Prüft ebenso die Eindeutigkeit der Mitgliedsnummer.
<code>findeMedium(int)</code>	Sucht mit <code>std::find_if</code> und Lambda; liefert <code>Medium*</code> oder <code>nullptr</code> .
<code>findeMitglied(int)</code>	Liefert <code>Mitglied*</code> oder <code>nullptr</code> .
<code>ausleihen(int, int)</code>	Prüft beide Objekte und ruft die Mitgliedsmethode auf.
<code>zurueckgeben(int, int)</code>	Leitet die Rückgabe an das gefundene Mitglied weiter.
<code>zeigeBestand() const</code>	Zeigt ID, Beschreibung und Verfügbarkeit.
<code>suche(const std::string&) const</code>	Liefert <code>std::vector<const Medium*></code> mit Titel- oder Urhebertreffern.

Eine Lambda-Funktion als Suchbedingung

```
auto it = std::find_if(medien.begin(), medien.end(),
    [id](const Medium& m) { return m.getId() == id; });
return it == medien.end() ? nullptr : &*it;
```

[id] übernimmt die gesuchte Nummer in die kleine Funktion. `find_if` liefert den ersten passenden Iterator. Bei `end()` gibt es keinen Treffer. `&*it` bildet nur bei einem Treffer die Adresse des gefundenen Elements.

Zeiger nach Änderungen neu ermitteln

Wenn ein Vektor seine Kapazität überschreitet, kann er einen neuen Speicherbereich belegen. Zeiger und Referenzen auf seine bisherigen Elemente werden dann ungültig. Das betrifft `Medium*`, `Mitglied*` und die Ergebniszeiger der Textsuche jeweils beim zugehörigen Vektor. [2]

```
b.mediumHinzufuegen(neuesMedium);
Medium* medium = b.findeMedium(1);
if (medium != nullptr) std::cout << *medium;
```

`const Medium*` verhindert eine Änderung über diesen Zeiger. Es verlängert nicht die Gültigkeit der Adresse. Im Hauptprogramm werden zunächst alle Objekte angelegt und erst danach kurzfristige Zeiger verwendet.

05 / AUFGABENBLATT 52

Bauen und Testergebnisse

Die Befehle werden im entpackten Projektordner bibliothek-cpp ausgeführt. Voraussetzung ist ein installierter C++17-fähiger g++-Compiler.

Linux und macOS mit g++

```
g++ -std=c++17 -Wall -Wextra src/main.cpp -o bibliothek
./bibliothek
```

Windows mit g++ und PowerShell

```
g++ -std=c++17 -Wall -Wextra src/main.cpp -o bibliothek.exe
.\bibliothek.exe
```

Tatsächlich geprüfte Fälle

Testgruppe	Anzahl	Ergebnis
Startzustand und erste Ausleihe	4	bestanden
Bereits verliehenes Medium	1	bestanden
Zweite und dritte Ausleihe sowie Limit	4	bestanden
Rückgabe und anschließende Ausleihe	6	bestanden
Unbekannte IDs und nullptr	6	bestanden
Titel-, Urheber-, Leer- und erfolglose Suche	4	bestanden
Gleichheit über die ID	1	bestanden
Ungültiger Titel und doppelte Kennungen	3	bestanden
Gesamt	29	29 bestanden

pruefe() erwartet eine wahre Bedingung und zählt erfolgreiche Tests. Wird eine Bedingung verletzt, wirft die Funktion eine Ausnahme. main() meldet den Fehler und endet mit Rückgabecode 1. Die Prüfungen bleiben auch mit -DNDEBUG aktiv.

Ausschnitt aus dem Programmlauf

29 Pruefungen bestanden.

AUSLEIHEN

Mia (101): 2 3 4

Noah (102): 1

Prüfumgebung: Linux, g++ 13.3.0, 02.10.2026. Zusätzliche Compilerprüfung mit -Wpedantic und -Werror: keine Warnungen. Die vollständige Programmausgabe steht in README.md.

Git und Abgabe

Die Dateien bilden die fachliche Musterlösung. Der GitHub-Nachweis entsteht durch den eigenen tatsächlichen Arbeitsverlauf. Das Aufgabenblatt verlangt mindestens sieben sinnvolle Commits, einen Feature-Branch mit Merge und ein offenes Issue.

Sinnvolle Etappen für echte Commits

Nr.	Beispiel für eine Commit-Nachricht
1	Projektstruktur und Build-Anleitung angelegt
2	Medium mit Enum und Validierung implementiert
3	Mitglied mit Ausleihlimit implementiert
4	Bibliothek mit ID-Suche und Ausleihe implementiert
5	Mediensuche und Vergleichsoperatoren ergaenzt
6	Hauptprogramm und automatische Pruefungen ergaenzt
7	Reflexion und Programmausgabe dokumentiert

Diese Liste ist eine Planung. Committe jeweils erst nach der eigenen Bearbeitung und Prüfung. Eine fertige Datei nachträglich in künstliche Entwicklungsschritte zu zerlegen, bildet keinen echten Arbeitsverlauf ab.

Feature-Branch für die Klasse Mitglied

```
git switch -c feature/mitglied
# Mitglied implementieren und pruefen, dann:
git add src/main.cpp
git commit -m "Mitglied mit Ausleihlimit implementiert"
git push -u origin feature/mitglied
```

Auf GitHub einen Pull Request nach main öffnen und mit einem Merge-Commit zusammenführen. Danach lokal auf main wechseln und mit `git pull --ff-only` aktualisieren. [3]

Abschluss nach erfolgreicher Prüfung

Ein offenes Issue kann die spätere Aufteilung der Klassen in Header- und Source-Dateien beschreiben. Zuerst alle Änderungen pushen, dann aus einem frischen Klon mit dem README-Befehl bauen und die Tests ausführen.

```
git tag v1.0
git push origin v1.0
```

Danach den Repository-Link abgeben. Bei einem privaten Repository benötigt die Lehrkraft Zugriff. Die vollständige Schrittfolge steht in GIT_ABGABE.md; ein Actions-Workflow bleibt optional. [4]

07 / AUFGABENBLATT 52

Reflexion

Musterantworten zu Teil 5. Die fachlichen Aussagen lassen sich am Code prüfen. Die persönliche Reflexion muss den eigenen Arbeitsprozess beschreiben.

1 Warum sind Attribute private

Private Attribute verhindern, dass anderer Code die Daten beliebig verändert. Getter erlauben gezielten lesenden Zugriff, während Methoden die erlaubten Zustandsänderungen kontrollieren. So kann der Titel nicht nachträglich unbemerkt auf einen leeren Text gesetzt werden.

2 Was bedeutet const bei Methoden

Eine const-Methode verändert die normalen Attribute des aufgerufenen Objekts nicht. Deshalb sind Getter, beschreibung(), hatAusgeliehen() und die Anzeige- und Textsuchmethoden als const markiert. Konsolenausgaben sind dabei möglich, weil sie den Objektzustand nicht verändern.

3 Warum können zurückgegebene Zeiger ungültig werden

findeMedium() liefert die Adresse eines Elements im Vektor. Wird beim Hinzufügen dessen Kapazität überschritten, kann der Vektor die Elemente in einen neuen Speicherbereich verschieben. Deshalb werden beim Mitglied IDs gespeichert und Medien bei Bedarf neu gesucht.

4 Welche Grenzen haben Enum und Switch

Ein Enum ist übersichtlich, solange die Medientypen dieselben Grunddaten besitzen. Für einen neuen Typ müssen die betroffenen switch-Anweisungen erweitert werden; besondere Daten wie eine DVD-Laufzeit passen nicht gut in die gemeinsame Klasse. Eigene Klassen können diese Unterschiede gezielter abbilden, machen das Modell aber aufwendiger.

5 Was ändert sich in einem größeren Projekt

Für diese Übung ist eine einzige Datei vorgeschrieben. In größeren Projekten enthalten Header üblicherweise die Klassendeklarationen und Source-Dateien die Methodendefinitionen. Das trennt Schnittstelle und Umsetzung, erleichtert die Zusammenarbeit und ermöglicht getrennte Übersetzung.

6 Wie wurde KI eingesetzt

Diese Musterlösung wurde mit KI erstellt; dazu gehören Code, Erklärungen und Testvorschläge. Für deine eigene Reflexion beschreibst du, welche Teile du anschließend selbst nachvollzogen und verändert hast. Einen konkreten Widerspruch oder eine Korrektur nennst du nur, wenn dies tatsächlich vorgekommen ist.

Abgabecheck und Quellen

Bereits im Projektpaket vorhanden

- ✓ Vollständiger C++17-Quellcode in src/main.cpp.
- ✓ README mit Build-Befehlen, Beispielausgabe und fünf Lernpunkten.
- ✓ REFLEXION.md mit sechs Antwortvorlagen und .gitignore.
- ✓ Alle fünf geforderten Testszenarien und insgesamt 29 Prüfungen.

Vor der eigenen Abgabe erledigen

- ☐ Code selbst erklären können und Reflexion persönlich anpassen.
- ☐ Repository bibliothek-cpp und mindestens sieben echte Commits.
- ☐ Feature-Branch bearbeiten und nach main zusammenführen.
- ☐ Ein offenes Issue für eine Zusatzaufgabe anlegen.
- ☐ Frischen Klon bauen und alle Prüfungen ausführen.
- ☐ Build- und IDE-Dateien aus dem Repository fernhalten.
- ☐ Tag v1.0 pushen und Repository-Link mit passendem Zugriff abgeben.

Quellen

[1] Dr. Michael Litera: Aufgabenblatt 52, Programmiertechnik, bereitgestellte PDF, Seiten 1 bis 5. Verbindliche Aufgaben und Abgabeanforderungen.

[2] C++ Working Draft, vector.modifiers: Reallokation und Ungültigkeit von Zeigern.
eel.is/c++draft/vector.modifiers

[3] Offizielle Git-Dokumentation: git switch. git-scm.com/docs/git-switch

[4] Offizielle Git-Dokumentation: git tag. git-scm.com/docs/git-tag

Onlinequellen geprüft am 02.10.2026. Der aktuelle C++-Entwurf dient hier ergänzend zur Erklärung der Zeigergültigkeit; der Programmcode verwendet C++17.

Designquelle: Julian Krauß, Corporate Identity & Design System, JK-CD-DE-2026-003, PDF-Revision 3.2, Release 4.6.3. Original-Logo und definierte Navy-, Blau- und Cyan-Farben.

09 / AUFGABENBLATT 52

Vollständiger Quellcode

src/main.cpp · Zeilen 1 bis 53 · Teil 1 von 6

```

001 // Aufgabenblatt 52: Bibliotheksverwaltung in C++17
002 // KI-gestuetzte Musterloesung zum Nachvollziehen und Anpassen.
003 #include <algorithm>
004 #include <iostream>
005 #include <stdexcept>
006 #include <string>
007 #include <vector>
008
009 enum class MediumTyp { Buch, DVD, Zeitschrift };
010
011 std::string typAlsText(MediumTyp typ) {
012     switch (typ) {
013         case MediumTyp::Buch: return "Buch";
014         case MediumTyp::DVD: return "DVD";
015         case MediumTyp::Zeitschrift: return "Zeitschrift";
016     }
017     throw std::invalid_argument("Ungueltiger Medientyp");
018 }
019
020 class Medium {
021 private:
022     int id;
023     std::string titel;
024     std::string urheber;
025     int jahr;
026     MediumTyp typ;
027     bool verfuegbar;
028
029 public:
030     Medium(int id, const std::string& titel,
031         const std::string& urheber, int jahr, MediumTyp typ)
032         : id(id), titel(titel), urheber(urheber), jahr(jahr),
033         typ(typ), verfuegbar(true) {
034         if (id <= 0 || jahr <= 0 ||
035             titel.find_first_not_of(" \t\r\n") == std::string::npos ||
036             urheber.find_first_not_of(" \t\r\n") == std::string::npos) {
037             throw std::invalid_argument("Ungueltige Mediendaten");
038         }
039         typAlsText(typ); // Prueft auch einen ungueltigen Enum-Wert.
040     }
041
042     int getId() const { return id; }
043     const std::string& getTitel() const { return titel; }
044     const std::string& getUrheber() const { return urheber; }
045     int getJahr() const { return jahr; }
046     MediumTyp getTyp() const { return typ; }
047     bool istVerfuegbar() const { return verfuegbar; }
048
049     bool ausleihen() {
050         if (!verfuegbar) return false;
051         verfuegbar = false;
052         return true;
053     }

```

09 / QUELLCODE

Quellcode Fortsetzung

src/main.cpp · Zeilen 54 bis 105 · Teil 2 von 6

```

054
055     bool zurueckgeben() {
056         if (verfuegbar) return false;
057         verfuegbar = true;
058         return true;
059     }
060
061     std::string beschreibung() const {
062         std::string verbindung;
063         switch (typ) {
064             case MediumTyp::Buch: verbindung = " von "; break;
065             case MediumTyp::DVD: verbindung = ", Regie: "; break;
066             case MediumTyp::Zeitschrift:
067                 verbindung = ", Verlag: "; break;
068         }
069         return typAlsText(typ) + ": " + titel + verbindung + urheber
070             + " (" + std::to_string(jahr) + ")";
071     }
072 };
073
074 std::ostream& operator<<(std::ostream& os, const Medium& m) {
075     return os << m.beschreibung();
076 }
077
078 bool operator==(const Medium& links, const Medium& rechts) {
079     return links.getId() == rechts.getId();
080 }
081
082 class Mitglied {
083 private:
084     std::string name;
085     int mitgliedsNr;
086     // IDs bleiben bei einer Reallokation des Medien-Vektors erhalten.
087     // Gespeicherte Zeiger oder Referenzen koennten ungueltig werden.
088     std::vector<int> ausgelieheneIds;
089
090 public:
091     Mitglied(const std::string& name, int mitgliedsNr)
092         : name(name), mitgliedsNr(mitgliedsNr) {
093         if (mitgliedsNr <= 0 ||
094             name.find_first_not_of(" \t\r\n") == std::string::npos) {
095             throw std::invalid_argument("Ungueltige Mitgliedsdaten");
096         }
097     }
098
099     const std::string& getName() const { return name; }
100     int getMitgliedsNr() const { return mitgliedsNr; }
101
102     bool hatAusgeliehen(int mediumId) const {
103         return std::find(ausgelieheneIds.begin(), ausgelieheneIds.end(),
104             mediumId) != ausgelieheneIds.end();
105     }

```

09 / QUELLCODE

Quellcode Fortsetzung

src/main.cpp · Zeilen 106 bis 154 · Teil 3 von 6

```

106
107     bool ausleihen(Medium& m) {
108         if (ausgelieheneIds.size() >= 3 || hatAusgeliehen(m.getId())) {
109             return false;
110         }
111         // Speicher reservieren, bevor der Medienstatus geaendert wird.
112         ausgelieheneIds.reserve(3);
113         if (!m.ausleihen()) return false;
114         ausgelieheneIds.push_back(m.getId());
115         return true;
116     }
117
118     bool zurueckgeben(Medium& m) {
119         auto it = std::find(ausgelieheneIds.begin(),
120                             ausgelieheneIds.end(), m.getId());
121         if (it == ausgelieheneIds.end() || !m.zurueckgeben()) {
122             return false;
123         }
124         ausgelieheneIds.erase(it);
125         return true;
126     }
127
128     void zeigeAusleihen() const {
129         std::cout << name << " (" << mitgliedsNr << "): ";
130         if (ausgelieheneIds.empty()) std::cout << "keine Ausleihen";
131         for (int id : ausgelieheneIds) std::cout << id << ' ';
132         std::cout << '\n';
133     }
134 };
135
136 class Bibliothek {
137 private:
138     std::vector<Medium> medien;
139     std::vector<Mitglied> mitglieder;
140
141 public:
142     void mediumHinzufuegen(Medium m) {
143         if (findeMedium(m.getId()) != nullptr) {
144             throw std::invalid_argument("Medien-ID bereits vorhanden");
145         }
146         medien.push_back(m);
147     }
148
149     void mitgliedHinzufuegen(Mitglied m) {
150         if (findeMitglied(m.getMitgliedsNr()) != nullptr) {
151             throw std::invalid_argument("Mitgliedsnummer vorhanden");
152         }
153         mitglieder.push_back(m);
154     }

```

09 / QUELLCODE

Quellcode Fortsetzung

src/main.cpp · Zeilen 155 bis 205 · Teil 4 von 6

```

155
156 // Zeiger nur kurzfristig nutzen: Hinzufuegen kann sie entwerten.
157 Medium* findeMedium(int id) {
158     auto it = std::find_if(medien.begin(), medien.end(),
159         [id](const Medium& m) { return m.getId() == id; });
160     return it == medien.end() ? nullptr : &*it;
161 }
162
163 Mitglied* findeMitglied(int mitgliedsNr) {
164     auto it = std::find_if(mitglieder.begin(), mitglieder.end(),
165         [mitgliedsNr](const Mitglied& m) {
166             return m.getMitgliedsNr() == mitgliedsNr;
167         });
168     return it == mitglieder.end() ? nullptr : &*it;
169 }
170
171 bool ausleihen(int mitgliedsNr, int mediumId) {
172     Mitglied* mitglied = findeMitglied(mitgliedsNr);
173     Medium* medium = findeMedium(mediumId);
174     if (mitglied == nullptr || medium == nullptr) return false;
175     return mitglied->ausleihen(*medium);
176 }
177
178 bool zurueckgeben(int mitgliedsNr, int mediumId) {
179     Mitglied* mitglied = findeMitglied(mitgliedsNr);
180     Medium* medium = findeMedium(mediumId);
181     if (mitglied == nullptr || medium == nullptr) return false;
182     return mitglied->zurueckgeben(*medium);
183 }
184
185 void zeigeBestand() const {
186     for (const Medium& m : medien) {
187         std::cout << m.getId() << " | " << m << " | "
188             << (m.istVerfuegbar() ? "frei" : "verliehen")
189             << '\n';
190     }
191 }
192
193 // Gross-/Kleinschreibung beachten; leerer Suchtext findet alles.
194 // Auch diese Ergebniszeiger nur bis zur naechsten Aenderung nutzen.
195 std::vector<const Medium*> suche(const std::string& suchtext) const {
196     std::vector<const Medium*> treffer;
197     for (const Medium& m : medien) {
198         if (m.getTitel().find(suchtext) != std::string::npos ||
199             m.getUrheber().find(suchtext) != std::string::npos) {
200             treffer.push_back(&m);
201         }
202     }
203     return treffer;
204 }
205 };

```

09 / QUELLCODE

Quellcode Fortsetzung

src/main.cpp · Zeilen 206 bis 254 · Teil 5 von 6

```

206
207 // Eigene Testfunktion: wird auch mit -DNDEBUG ausgefuehrt.
208 void pruefe(bool bedingung, const std::string& text, int& anzahl) {
209     if (!bedingung) throw std::runtime_error("Test fehlgeschlagen: " + text);
210     ++anzahl;
211     std::cout << "[OK] " << text << '\n';
212 }
213
214 int main() {
215     try {
216         Bibliothek b;
217         b.mediumHinzufuegen({1, "C++ Grundlagen", "Anna Weber", 2024,
218             MediumTyp::Buch});
219         b.mediumHinzufuegen({2, "Netzwerke", "Ben Wolf", 2023,
220             MediumTyp::Buch});
221         b.mediumHinzufuegen({3, "IT im Alltag", "Clara Berg", 2022,
222             MediumTyp::DVD});
223         b.mediumHinzufuegen({4, "Technik heute", "Tech Verlag", 2025,
224             MediumTyp::Zeitschrift});
225         b.mediumHinzufuegen({5, "Linux Praxis", "Anna Weber", 2024,
226             MediumTyp::Buch});
227         b.mitgliedHinzufuegen({"Mia", 101});
228         b.mitgliedHinzufuegen({"Noah", 102});
229
230         std::cout << "BIBLIOTHEK | Bestand vor den Tests\n";
231         b.zeigeBestand();
232         std::cout << "\nAUTOMATISCHE PRUEFUNGEN\n";
233         int tests = 0;
234         pruefe(b.findeMedium(1)->istVerfuegbar(),
235             "Medium anfangs frei", tests);
236         pruefe(b.ausleihen(101, 1), "Erste Ausleihe erfolgreich", tests);
237         pruefe(!b.findeMedium(1)->istVerfuegbar(), "Status verliehen", tests);
238         pruefe(b.findeMitglied(101)->hatAusgeliehen(1),
239             "ID gespeichert", tests);
240         pruefe(!b.ausleihen(102, 1), "Verliehenes Medium abgelehnt", tests);
241         pruefe(b.ausleihen(101, 2), "Zweite Ausleihe erfolgreich", tests);
242         pruefe(b.ausleihen(101, 3), "Dritte Ausleihe erfolgreich", tests);
243         pruefe(!b.ausleihen(101, 4), "Viertes Medium abgelehnt", tests);
244         pruefe(b.findeMedium(4)->istVerfuegbar(),
245             "Abgelehntes Medium frei", tests);
246         pruefe(!b.zurueckgeben(102, 1), "Fremde Rueckgabe abgelehnt", tests);
247         pruefe(b.zurueckgeben(101, 1), "Eigene Rueckgabe erfolgreich", tests);
248         pruefe(!b.findeMitglied(101)->hatAusgeliehen(1),
249             "ID entfernt", tests);
250         pruefe(!b.zurueckgeben(101, 1),
251             "Doppelte Rueckgabe abgelehnt", tests);
252         pruefe(b.ausleihen(102, 1), "Anderes Mitglied leiht erneut", tests);
253         pruefe(b.ausleihen(101, 4), "Nach Rueckgabe wieder Platz", tests);
254         pruefe(!b.ausleihen(999, 5), "Unbekanntes Mitglied abgelehnt", tests);

```

09 / QUELLCODE

Quellcode Fortsetzung

src/main.cpp · Zeilen 255 bis 299 · Teil 6 von 6

```

255     pruefe(!b.ausleihen(101, 999), "Unbekanntes Medium abgelehnt", tests);
256     pruefe(!b.zurueckgeben(999, 1),
257           "Rueckgabe mit unbekanntem Mitglied", tests);
258     pruefe(!b.zurueckgeben(101, 999),
259           "Rueckgabe mit unbekanntem Medium", tests);
260     pruefe(b.findeMedium(999) == nullptr,
261           "Mediensuche liefert nullptr", tests);
262     pruefe(b.findeMitglied(999) == nullptr,
263           "Mitgliedssuche liefert nullptr", tests);
264     pruefe(b.suche("Linux").size() == 1, "Titelsuche", tests);
265     pruefe(b.suche("Anna Weber").size() == 2, "Urhebersuche", tests);
266     pruefe(b.suche("xyz").empty(), "Suche ohne Treffer", tests);
267     pruefe(b.suche("").size() == 5,
268           "Leere Suche findet alle Medien", tests);
269     Medium vergleich(1, "Anderer Titel", "Andere Person", 2020,
270           MediumTyp::DVD);
271     pruefe(*b.findeMedium(1) == vergleich, "Gleichheit ueber ID", tests);
272
273     bool fehler = false;
274     try { Medium unguelteig(6, " ", "Autor", 2024, MediumTyp::Buch); }
275     catch (const std::invalid_argument&) { fehler = true; }
276     pruefe(fehler, "Leerer Titel wirft invalid_argument", tests);
277     fehler = false;
278     try { b.mediumHinzufuegen(vergleich); }
279     catch (const std::invalid_argument&) { fehler = true; }
280     pruefe(fehler, "Doppelte Medien-ID wirft Fehler", tests);
281     fehler = false;
282     try { b.mitgliedHinzufuegen({"Andere Person", 101}); }
283     catch (const std::invalid_argument&) { fehler = true; }
284     pruefe(fehler, "Doppelte Mitgliedsnummer wirft Fehler", tests);
285
286     std::cout << '\n' << tests << " Pruefungen bestanden.\n\n";
287     std::cout << "AUSLEIHEN\n";
288     b.findeMitglied(101)->zeigeAusleihen();
289     b.findeMitglied(102)->zeigeAusleihen();
290     std::cout << "\nSUCHE nach Anna Weber\n";
291     for (const Medium* m : b.suche("Anna Weber")) std::cout << *m << '\n';
292     std::cout << "\nBESTAND nach den Tests\n";
293     b.zeigeBestand();
294 } catch (const std::exception& e) {
295     std::cerr << "Fehler: " << e.what() << '\n';
296     return 1;
297 }
298 return 0;
299 }
```